MULT20015 Elements of Quantum Computing

Practice Class 5

Welcome to Practice Class-5 of MULT20015 Elements of Quantum Computing, covering exercises relevant to the material presented in lectures (9 and 10) in Week 5.

The purpose of this week's exercises is to:

- understand complexity classes of simple classical algorithms
- simulate and understand Deutsch-Jozsa algorithm by implementing it on QUI
- simulate and understand Simon's algorithm by implementing it on QUI

The exercises in these notes are to assist your understanding of the subject and may require time outside of the practice class to complete.

1. Complexity classes for simple classical algorithms

Algorithms are systematic sets of instructions which when performed on an input data set, produce an output data set. Algorithms are designed to solve computational problems on computers. The efficiency of an algorithm is the measure of its execution time, and is often represented as a function $f(n)$ describing the number of elementary steps for a problem of size described by n. The algorithms are classified in Big O and Big Omega classes as you have learned in lecture 9.

In this section, you will practice how to compute $f(n)$ for a given algorithm and how to classify an algorithm defined by $f(n)$ in terms of Big O and Big Omega notations.

Exercise 1.1 In lecture 9, you have learnt the insertion sort algorithm which takes $O(n^2)$ steps under the worst case to sort an array of "n" number of elements. Here, you are going to analyse a slightly different sorting algorithm, as described below:

Input = An unsorted array of "n" integers.
Output = A sorted array with all elements arranged in ascending order.

Algorithm: repeatedly swap adjacent elements if they are in wrong order, i.e. left element is larger than the right element. The algorithm exits when all elements are sorted, and no swap between adjacent elements occurs.

Example Input array: <u>4, 3,</u> 5
Algorithm steps:
4>3: swap 4 and 3 → 3, <u>4, 5</u>
4<5: no swap → 3, 4, 5
Exit.

Perform the above algorithm on the following input array. Show all steps.

Input array: 34, 45, 23, 87, 98, 12, 11, 89, 78, 56, 11

The elementary step for the above algorithm is comparison between two elements of the array. Compute the worst-case cost i.e. $f(n)$ of the algorithm:

What is the elementary step? _____

How many elementary steps are required for an input array of size n in worst case?

What's the Big O class for the above algorithm? _____

What's the Big Omega class for the above algorithm? _____

Exercise 1.2 In this exercise, you will practice more about calculation of Big O and Big Omega classes for given $f(n)$ functions.

Recall: $f(n) = O(g(n))$ when for some c > 0 and large n_0, $f(n) \leq c \times g(n)$ if $n \geq n_0$.
$f(n) = \Omega(g(n))$ when for some c > 0 and large n_0, $f(n) \geq c \times g(n)$ if $n \geq n_0$.

For each of the $f(n)$ functions given below, find and plot $g(n)$ and compute n_0 and c.

$f(n) = 9n + 5$

$f(n) = 3n^2 + 10^6 n + 10^{10}$

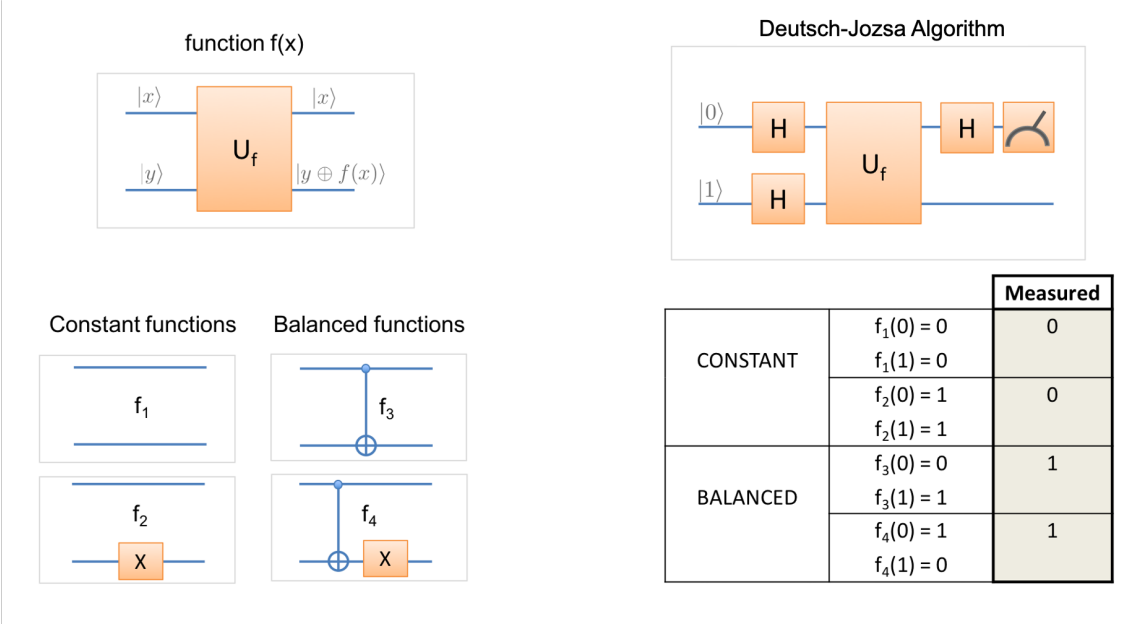
$f(n) = n\log_2(n) + n^2$

$f(n) = 2^n + n^{100}$

Note: try to make the bounds as tight as possible.

2. Deutsch-Jozsa Algorithm

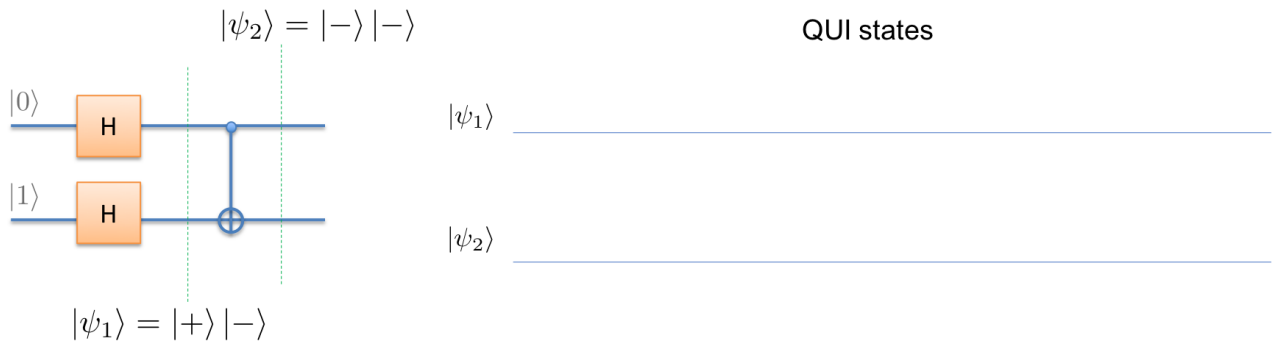
The Deutsch-Jozsa algorithm is a quantum algorithm which requires only one query to determine if a function is constant or balanced. Classically, this task will require $O(2^n)$ queries. The purpose of this section is to implement and understand the Deutsch-Jozsa Algorithm on the Quantum User Interface (QUI).



Exercise 2.1

a) Program the Deutsch-Jozsa circuit in the QUI as per above and verify the algorithm outputs for the four function types.

b) Examine the following circuit in the QUI – program the following circuit (make sure you understand the X-basis states shown) and fill in the QUI states as indicated (ket/binary representation). Can you see the working of the circuit in the QUI representation?

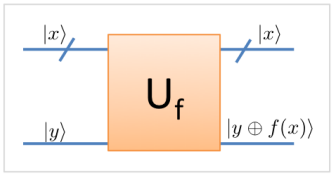


Exercise 2.2 For the case of a balanced function (case 4) extend the Deutsch-Jozsa circuit coded in the QUI to multiple (two) qubits. Along the way ensure you understand how the circuit is working by careful inspection of the states at each time step.

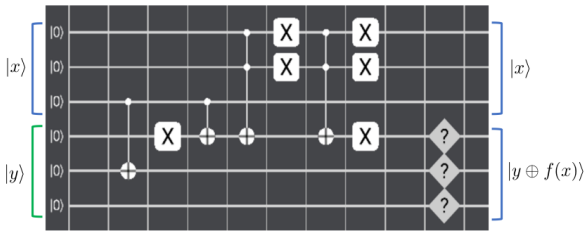
3. Simon's Algorithm

Here we will implement a 3-bit example of finding a 2-to-1 Boolean function, i.e. $a: f(x) = f(x \oplus a)$ (where the addition is bitwise addition modulo 2), using Simon's algorithm.

Exercise 3.1 Consider the following circuit which implements an oracle function $f(x)$ over 3-bit numbers, with a period a.



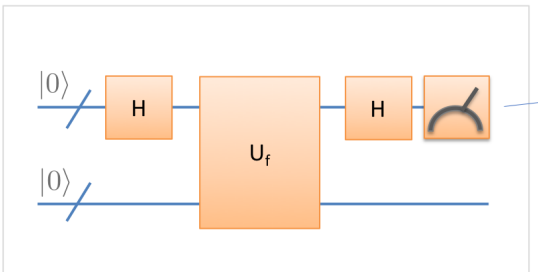
Oracle function f(x)



x	f(x)
000	
001	
010	
011	
100	
101	
110	
111	

a) Program and run the circuit and fill in the table. By (classical) inspection, what is the period a of the function we wish to determine (both bit and decimal forms)?

b) Incorporate the oracle into a circuit in the QUI to determine the period using Simon's algorithm. Run the circuit several times to obtain three distinct integers (y_a, y_b, y_c) in the upper register. From the system of equations (as per schematic below) and solve by hand (a classical step) to obtain the period a in bit form, and hence determine the decimal (integer) form of a and compare with the classical solution above.



Measurement on upper register gives results, y:

$$y: \ a \cdot y = 0 \mod 2$$

Solve equations to determine the period, a.

a in bit form

1st y in bit form					a_0		0
2nd y in bit form				x	a_1	=	0
3rd y in bit form					a_2		0

multiplication mod 2

c) Run the time scrubber through the circuit and see if you can understand how it's working.

Exercise 3.2 Here you will create another instance of a periodic function over 3 bits for which you can apply Simon's algorithm, essentially repeating the steps of Exercise 3.1.

x	f(x)
000	
001	
010	
011	
100	
101	
110	
111	

Fill in the table for your function (left) and create the corresponding "oracle" circuit (right). Code into the QUI and follow similar steps as in Exercise 3.1 above to extract the required data from the QC to then classically solve for the period.

Exercise 3.3 See if you can create a 4-qubit instance (or even higher!).